



Escuela Técnica Superior de
Ingeniería Informática

SEGURIDAD EN SISTEMAS INFORMÁTICOS Y EN INTERNET

PAI-2.

**BYODSEC-BRING YOUR OWN DEVICE SEGURO PARA UNA
UNIVERSIDAD PÚBLICA USANDO ROAD WARRIOR VPN SSL**

SECURITY TEAM 2

Alejandro Fernández Orozco
Alejandro Morilla García

1. CANAL DE COMUNICACIÓN SEGURO	3
1.1 PROBLEMA PLANTEADO	3
1.2 SOLUCIÓN APORTADA	4
1.3 OBJETIVOS	4
2. DESARROLLO, PRUEBAS Y ANÁLISIS DE LOS RESULTADOS	4
2.1 COMUNICACIÓN CLIENTE/SERVIDOR	4
2.2 CREACIÓN DE CERTIFICADOS EN OPENSSL	5
2.3 ANÁLISIS DEL TRÁFICO	5
3. CONCLUSIONES	6

RESUMEN EJECUTIVO

En este proyecto hemos implementado un sistema de Bring Your Own Device seguro para los empleados de una universidad. Para ello hemos necesitado generar una clave privada, una solicitud de certificado y un certificado firmado para su uso en TLS. Hemos implementado el servicio con estos 3 elementos y con los siguientes archivos python:

- Cliente: se comunica con un servidor a través de una conexión de red, con IP y puerto especificados. Configuramos el contexto SSL del cliente con los 3 elementos mencionados anteriormente.
- Servidor: acepta conexiones seguras de clientes, recibe mensajes JSON con las credenciales de cada usuario y manda un mensaje de confirmación. Utiliza TLS para garantizar que las credenciales se transmitan de forma segura.
- Carga-300: Establece el número de conexiones seguras que tenemos que probar para ver el correcto funcionamiento del servidor . Hace lo mismo que el cliente pero automatizado, de forma que realiza 300 conexiones al mismo tiempo.

Este sistema nos ayuda a guardar claves de usuario y contraseñas de forma cifrada en un almacén de datos seguro, gracias a la clave privada y a los certificados TLS.

1. CANAL DE COMUNICACIÓN SEGURO

1.1 PROBLEMA PLANTEADO

En este proyecto, se nos solicita la implementación de la Política de Seguridad Bring your Own Device (BYOD) seguro para los empleados de una Universidad Pública. Para la transmisión de todos los datos, es fundamental la implementación de canales de comunicación seguros. Una de las maneras para crear canales de comunicación seguros es la utilización de VPN (Virtual Private Networks) que usan en su implementación Sockets.

La información puede transmitirse en “claro” o cifrada dependiendo del tipo de Socket que se utilice o de los algoritmos que se utilicen con la información a transmitir . La política de seguridad de la organización nos dice con respecto a la seguridad en las transmisiones de información cliente-servidor: “... deberían ser confidenciales e íntegras y además autenticadas.”

También se nos plantea que tengamos en cuenta que el número de empleados que usarán la aplicación será aproximadamente de unos 300. Será recomendable una herramienta que permita comprobar la comunicación segura entre equipos.

1.2 SOLUCIÓN APORTADA

Siguiendo la Política de Seguridad que se nos exigía, la solución más conveniente ha sido simular una conexión cliente-servidor donde podamos enviar el mensaje con el usuario y la contraseña de acceso a cualquier sitio web, y que la reciba el servidor, con la condición de que tengan certificados para comunicarse. Hemos establecido el protocolo TLS 1.3 para mayor seguridad.

1.3 OBJETIVOS

- Desarrollar un sistema cliente-servidor basado en certificados generados por OpenSSL de acuerdo con lo exigido en la política de seguridad.
- Se debe razonar y demostrar la eficiencia de la solución aportada, y que se permita para un número de empleados determinado.

2. DESARROLLO, PRUEBAS Y ANÁLISIS DE LOS RESULTADOS

2.1 COMUNICACIÓN CLIENTE/SERVIDOR

El programa que vamos a detallar ha sido desarrollado en lenguaje Python, y se ha usado Visual Studio Code como entorno de desarrollo del programa. Hemos elegido el lenguaje Python porque nos proporciona una gran cantidad de módulos relacionados con la seguridad informática y la creación fácil de un socket:

La estructura del programa, en general, consta de dos archivos principales y otro que simula la conexión de 300 usuarios a la vez:

- **client-socket.py**: archivo donde hemos desarrollado el código de un cliente que se comunica con el servidor a través de una conexión red, con la dirección IP y puertos especificados. Es necesario configurar el contexto SSL del cliente para autenticar al servidor, requerir un certificado y establecer TLS 1.3
- **server-socket.py**: archivo que crea un servidor que acepta conexiones seguras de clientes, interpreta mensajes JSON que contiene credenciales de usuario, y responde con un mensaje de confirmación. El servidor utiliza TLS para asegurar la comunicación y garantizar que las credenciales se transmitan de manera segura.
- **carga300.py**: este es el archivo responsable de establecer conexiones seguras con el servidor . Primero, crea un contexto SSL para garantizar la seguridad de la comunicación, establece una conexión TCP con el servidor y muestra la versión de TLS utilizada. Posteriormente, genera un nombre de usuario y contraseña únicos, los convierte en un mensaje JSON y lo envía al servidor . La función espera una respuesta

del servidor, que se imprime en la consola. La función ejecuta las conexiones de forma paralela simulando así 300 conexiones simultáneas al servidor.

2.2 CREACIÓN DE CERTIFICADOS EN OPENSSSL

Necesitaremos generar una clave privada, una solicitud de certificado y un certificado firmado para su uso en TLS. Estos certificados son fundamentales para la autenticación y el cifrado de las comunicaciones seguras en Internet, como las que se utilizan en sitios web seguros (HTTPS) y otros servicios que requieren seguridad en la capa de transporte como TLS, para ello utilizamos estos 3:

- **openssl genpkey -algorithm RSA -out key.pem -aes256:** Este comando genera un par de claves asimétricas RSA y las almacena en un archivo llamado key.pem. Las claves se cifran con AES-256 para protegerlas. Este par de claves se utiliza comúnmente en TLS para autenticación y cifrado. (Fig. 1)
- **openssl req -new -key key.pem -out PAI2.pem -config csr.cnf:** Este comando crea una solicitud de certificado (CSR, Certificate Signing Request) a partir de las claves generadas anteriormente. La solicitud de certificado se almacena en un archivo llamado PAI2.pem. El archivo **csr.cnf** (Fig. 2) se utiliza para configurar los detalles de la solicitud, como el nombre del sujeto y otros atributos del certificado. (Fig. 3)
- **openssl x509 -signkey key.pem -in PAI2.pem -req -days 365 -out CPAI2.pem -extfile csr.cnf -extensions req_ext:** Este comando toma la solicitud de certificado generada en el paso anterior (PAI2.pem) y firma un certificado X509 con las claves privadas del paso 1 (key.pem). El certificado firmado se almacena en un archivo llamado CPAI2.pem. El archivo csr.cnf se utiliza nuevamente para especificar las extensiones y detalles del certificado, y se utiliza nuevamente para especificar las extensiones y detalles del certificado, y se utiliza un período de validez de 365 días. (Fig. 4)

2.3 ANÁLISIS DEL TRÁFICO

Hemos usado Wireshark (Fig. 5), y tráfico local en loopback. Hemos obtenido lo siguiente:

En la Figura 6 podemos apreciar la trama del mensaje, en el cual podemos apreciar que no podemos obtener los datos de nombre y contraseña, ya que está cifrado. Para hacer que estos campos sean visibles, tendríamos que tener una conexión con protocolos no seguros, y saldría el texto en claro en el seguimiento del flujo TCP.

Aunque usemos el protocolo SSL/TLS, es posible analizar el algoritmo de Cipher Suite utilizado en la comunicación. En el paquete “Server Hello”, y desplegando el campo TLS, podemos ver que en Cipher Suite que contiene la información “TLS AES 25 AES GCM SHA384”, lo que indica que ese es el algoritmo utilizado en la conversación para cifrar el contenido. (Fig. 7)

Para la prueba del funcionamiento del **carga300.py**, hemos vuelto a usar Wireshark y hemos obtenido lo siguiente:

En la figura 8 vemos que hemos obtenido un total de 6270 tramas, lo que nos permite comprobar que la carga de las 300 conexiones simultáneas ha sido un éxito. Además por la consola de visual studio, que nos indica que ha habido 300 conexiones.(Fig. 9)

En la ultima trama de cada conexión podemos ver que la información que queremos enviar al servidor ha sido enviada de forma encriptada gracias al protocolo TLS 1.3 (Fig. 10)

3. CONCLUSIONES

Hemos concluido el proyecto con éxito, logrando implementar un **sistema cliente-servidor seguro** siguiendo las pautas de la **política de seguridad especificada**, con **certificados TLS** y una **clave privada** generada por **OpenSSL**.

Nuestro sistema se compone de:

- Cliente
- Servidor
- Un script de carga para simular **300 conexiones simultáneas**

El proceso de generación de **claves privadas** y **certificados** mediante **OpenSSL** ha sido crucial para garantizar la autenticación y el cifrado de las comunicaciones seguras.

El análisis de tráfico con **Wireshark** confirma que los datos se transmiten de manera cifrada, demostrando la eficacia del **protocolo TLS 1.3**. Además, el script de carga **carga300.py** demuestra la capacidad del servidor para soportar múltiples conexiones simultáneas. Este análisis destaca la importancia de seguir **prácticas de seguridad de red** al implementar soluciones **BYOD**, resaltando la utilidad de **TLS** y **OpenSSL** para conexiones seguras.

Por todo esto, podemos afirmar que hemos creado una **solución efectiva y segura**, un sistema que garantiza la seguridad y la integridad de los datos transmitidos, cumpliendo la **política de seguridad de la organización**, además de asegurar la **escalabilidad** con una prueba exitosa de **300 usuarios simultáneos**.

(Fig. 1)

(Fig. 2)

(Fig. 3)

(Fig. 4)

captura_final.pcapng

Archivo Edición Visualización Jr Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

tcp.stream eq 0

No.	Time	Source	Destination	Protocol	Length	Info
7	8.333846	192.168.1.147	192.168.1.147	TCP	56	61396 → 3030 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
8	8.333999	192.168.1.147	192.168.1.147	TCP	56	3030 → 61396 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
9	8.333938	192.168.1.147	192.168.1.147	TCP	44	61396 → 3030 [ACK] Seq=1 Ack=1 Win=2161152 Len=0
10	8.334505	192.168.1.147	192.168.1.147	TLSv1.3	269	Client Hello
11	8.334546	192.168.1.147	192.168.1.147	TCP	44	3030 → 61396 [ACK] Seq=1 Ack=226 Win=2160896 Len=0
12	8.336437	192.168.1.147	192.168.1.147	TLSv1.3	1596	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data
13	8.336494	192.168.1.147	192.168.1.147	TCP	44	61396 → 3030 [ACK] Seq=226 Ack=1553 Win=2159616 Len=0
14	8.337331	192.168.1.147	192.168.1.147	TLSv1.3	124	Change Cipher Spec, Application Data
15	8.337372	192.168.1.147	192.168.1.147	TCP	44	3030 → 61396 [ACK] Seq=1553 Ack=306 Win=2160896 Len=0
16	8.337678	192.168.1.147	192.168.1.147	TLSv1.3	299	Application Data
17	8.337768	192.168.1.147	192.168.1.147	TCP	44	61396 → 3030 [ACK] Seq=306 Ack=1808 Win=2159360 Len=0
18	8.337962	192.168.1.147	192.168.1.147	TLSv1.3	299	Application Data
19	8.338019	192.168.1.147	192.168.1.147	TCP	44	61396 → 3030 [ACK] Seq=306 Ack=2063 Win=2159104 Len=0
20	16.877268	192.168.1.147	192.168.1.147	TLSv1.3	118	Application Data
21	16.877349	192.168.1.147	192.168.1.147	TCP	44	3030 → 61396 [ACK] Seq=2063 Ack=380 Win=2160896 Len=0
22	16.878193	192.168.1.147	192.168.1.147	TLSv1.3	86	Application Data
23	16.878252	192.168.1.147	192.168.1.147	TCP	44	61396 → 3030 [ACK] Seq=380 Ack=2105 Win=2159104 Len=0

Frame 15: 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface \Device\NPF_{...} ...

Internet Protocol Version 4, Src: 192.168.1.147, Dst: 192.168.1.147

Transmission Control Protocol, Src Port: 3030, Dst Port: 61396, Seq: 1553, Ack: 306, Len: 0

Source Port: 3030

Destination Port: 61396

[Stream index: 0]

[Stream Packet Number: 0]

[Conversation completeness: Incomplete, DATA (15)]

[TCP Segment Len: 0]

Sequence Number: 1553 (relative sequence number)

Sequence Number (raw): 2779714497

[Next Sequence Number: 1553 (relative sequence number)]

Acknowledgment Number: 306 (relative ack number)

Acknowledgment number (raw): 2107142737

0101 = Header Length: 20 bytes (5)

Flags: 0x010 (ACK)

Window: 8441

[Calculated window size: 2160896]

[Window size scaling factor: 256]

Checksum: 0x655f [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

The completeness of the conversation capture (tcp.completeness)

Paquetes: 29 - Displayed: 17 (58.6%) - Perdido: 0 (0.0%)

Perfil: Default

(Fig. 5)

Wireshark - Seguir secuencia TCP (tcp.stream eq 0) - Adapter for loopback traffic capture

.....

R.....0.J.&...].M.N.<{...Pd .p....

... (ce.z...3=...q.....

.....#.....

.....+.....3.&\$.J6. |...*

..=...\$.I".P.....

...Z...v...Y.lj...k.U...`y...p...A.X .p....

... (ce.z...3=...q.....+.....3.\$\$. OMH...-y..... | #HC.....FS.....o?.#j.....@*.ty...`.....q{...d.^!....

l9<N.....m.K...0j]...BN>.....b.X.yss.....

0K.f...<.j.^T&.../...|...h.k.R.K.jA...Q.....k.i.Eah.q...R'.?..G..F.U.px...F.KP.h.\$.....M..O.....'.g..2*..

..0..%.bc..G.8.]J.F.t.v.Z..H'd##...;e*.U.....nt..Ifni.Z.#.H...O...e..E\nU.Og.....6r.Q.1.....wZ.b...e..R.

Zj.....M.....M..f.)/.W.I .j;.....?..]..5.....g>v.{.dy.../..U.3.....(z5.....4...4...N.|.....a.8.\$..(8.H.

... ^|<^..Ph...tZ.....Y)ZL.....I=..1B..c.H.IQ.1.?a.Q...u.'w.'^c.N.....RA2.....P....e.....A.tK.....!.....J..n..id..

...D .0?.6.z..

..2.H..S.v...C>...@.9...t..j..

1k...Hc...kepK...0.....Y.">.r.m.h|XtY. .:..0...c.e#lZ..k.....J(...~E.L".*...;=CcC.....c.XX..T

\$.KV.u.*...C[1..DU.O.P...aU.m.SN..1.....Oa.%L{.bl..Vb..k.....3..b..|T@c...c..W.....86H.....8...~.....r....

..lM.X_...#...e(eyY...r)(Ide.T..0..gp.DMp... (p..w.k...X.....

A9.....Pre.B% .0..iv.w~t.\..r]i....." +=...aB^..^k...G.....3.....2..C...^.....M<...~...J....V..T

...J.Y...w..j.^/~b..Ge|..1...4.....

S..d.7...0.N.5.b...1.Z.....^.....C...s.3lp{o.\(..b...c./)m-.3...I..V....., #^...X#HP.1...G.....^

...N.....~C...).9Jn0.>p.+x...3X_#.ub...v.].../.Y\$Z...a.....

2...t..R...v.2.2...9.X|+. {.....AP.....IF.Xg.....E..... z28..-Rv..wCZR.....G|). [z..%.az.\...6R.G(>....

^.....

..Fz.....

.....E...J..b.....J4.<...n{[.V.11..|..@.07...>..%9.v8..g@.....(..k.....

.....&.18A:.....

..e.nS1zA.IE..f/..V <#1...S..P...UJ#.....

.....?M..j..].....

..xw..q<...c...U.....KVZ1..+'U..x..@w.\DC..d"b--R=?.....5...S.....1.v... ..G.....WG.>.(.1.....|!I.....h..'=A..

<U..wb.J..N(.e_6M.....?..#Rk.9Yg.....?b.{.H.p.Z]...u.P7Y.....C.u...U.1..5K.....a..R.e@>E..U.k8..d...

M.J..f.6(x...e.l)u..0...QH.t'.{S.....}.:8.....

..C.f.....gftCI.....k.n*...ab..2+). yj.7..Cs.....+y..U7..a.....+.3s(.p*.B.....g.)h.....P....WF...Z...x.h.7..b..

.....{..1ZK.....

...E[...6..^.....j'2.n..2...).q.DWT}.....&.....X...\$.U.T. :).P.]..

4c.V.....

...%.E..?m...U...(. ..

tKf.l.6..o.gB.....

(Fig. 6)

capture_final.pcapng

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

tcpstream eq 0

No.	Time	Source	Destination	Protocol	Length	Info
7	8.333846	192.168.1.147	192.168.1.147	TCP	56	61396 → 3030 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
8	8.333999	192.168.1.147	192.168.1.147	TCP	56	3030 → 61396 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
9	8.333938	192.168.1.147	192.168.1.147	TCP	44	61396 → 3030 [ACK] Seq=1 Ack=1 Win=2161152 Len=0
10	8.334595	192.168.1.147	192.168.1.147	TLSv1.3	269	Client Hello
11	8.334546	192.168.1.147	192.168.1.147	TCP	44	3030 → 61396 [ACK] Seq=1 Ack=226 Win=2160896 Len=0
12	8.336437	192.168.1.147	192.168.1.147	TLSv1.3	1596	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data
13	8.336494	192.168.1.147	192.168.1.147	TCP	44	61396 → 3030 [ACK] Seq=226 Ack=1553 Win=2159616 Len=0
14	8.337331	192.168.1.147	192.168.1.147	TLSv1.3	124	Change Cipher Spec, Application Data
15	8.337372	192.168.1.147	192.168.1.147	TCP	44	3030 → 61396 [ACK] Seq=1553 Ack=306 Win=2160896 Len=0
16	8.337678	192.168.1.147	192.168.1.147	TLSv1.3	299	Application Data
17	8.337768	192.168.1.147	192.168.1.147	TCP	44	61396 → 3030 [ACK] Seq=306 Ack=1808 Win=2159360 Len=0
18	8.337962	192.168.1.147	192.168.1.147	TLSv1.3	299	Application Data
19	8.338019	192.168.1.147	192.168.1.147	TCP	44	61396 → 3030 [ACK] Seq=306 Ack=2063 Win=2159104 Len=0
20	16.877268	192.168.1.147	192.168.1.147	TLSv1.3	118	Application Data
21	16.877349	192.168.1.147	192.168.1.147	TCP	44	3030 → 61396 [ACK] Seq=2063 Ack=380 Win=2160896 Len=0
22	16.878193	192.168.1.147	192.168.1.147	TLSv1.3	86	Application Data
23	16.878252	192.168.1.147	192.168.1.147	TCP	44	61396 → 3030 [ACK] Seq=380 Ack=2105 Win=2159104 Len=0

Frame 12: 1596 bytes on wire (12768 bits), 1596 bytes captured (12768 bits) on interface (Device)

Null/Loopback

Internet Protocol Version 4, Src: 192.168.1.147, Dst: 192.168.1.147

Transmission Control Protocol, Src Port: 3030, Dst Port: 61396, Seq: 1, Ack: 226, Len: 1552

Transport Layer Security

TLSv1.3 Record Layer: Handshake Protocol: Server Hello

Content Type: Handshake (22)

Version: TLS 1.2 (0x0303)

Length: 122

Handshake Protocol: Server Hello

Handshake Type: Server Hello (2)

Length: 118

Version: TLS 1.2 (0x0303)

Random: ab59866c6abdc5e7db6bd558913c4146079a0bfeca5702ed1c4b841c9a758

Session ID Length: 32

Session ID: 0f70addd0fb00ae41bc2d863658f71abff799d7333de2aafec0bd871d584

Cipher Suite: TLS_AES_256_GCM_SHA384 (0x1302)

Compression Method: null (0)

Extensions Length: 46

Extension: supported_versions (Len=2) TLS 1.3

Extension: key_share (Len=36) x25519

[JA3S: Fullstr: 771,4866,43-51]

[JA3S: 15af977ce2de452b99a6fa2addb1036]

The completeness of the conversation capture (tcp.completeness)

Packets: 29 - Displayed: 17 (58.6%) - Perdido: 0 (0.0%)

Perfil: Default

(Fig. 7)

*Adapter for loopback traffic capture

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

tcpport == 3030

No.	Time	Source	Destination	Protocol	Length	Info
6175	40.777827	192.168.1.147	192.168.1.147	TCP	44	62188 → 3030 [RST, ACK] Seq=391 Ack=2129 Win=0 Len=0
6176	40.780601	192.168.1.147	192.168.1.147	TCP	56	62188 → 3030 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
6177	40.780689	192.168.1.147	192.168.1.147	TCP	56	3030 → 62188 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
6178	40.780722	192.168.1.147	192.168.1.147	TCP	44	62188 → 3030 [ACK] Seq=1 Ack=1 Win=2161152 Len=0
6179	40.781247	192.168.1.147	192.168.1.147	TLSv1.3	269	Client Hello
6180	40.781282	192.168.1.147	192.168.1.147	TCP	44	3030 → 62188 [ACK] Seq=1 Ack=226 Win=2160896 Len=0
6181	40.782218	192.168.1.147	192.168.1.147	TLSv1.3	1596	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data
6182	40.782269	192.168.1.147	192.168.1.147	TCP	44	62188 → 3030 [ACK] Seq=226 Ack=1553 Win=2159616 Len=0
6183	40.782856	192.168.1.147	192.168.1.147	TLSv1.3	124	Change Cipher Spec, Application Data
6184	40.782882	192.168.1.147	192.168.1.147	TCP	44	3030 → 62188 [ACK] Seq=1553 Ack=306 Win=2160896 Len=0
6185	40.782993	192.168.1.147	192.168.1.147	TLSv1.3	128	Application Data
6186	40.783009	192.168.1.147	192.168.1.147	TCP	44	3030 → 62188 [ACK] Seq=1553 Ack=390 Win=2160896 Len=0
6187	40.783108	192.168.1.147	192.168.1.147	TLSv1.3	299	Application Data
6188	40.783135	192.168.1.147	192.168.1.147	TCP	44	62188 → 3030 [ACK] Seq=390 Ack=1808 Win=2159360 Len=0
6189	40.783198	192.168.1.147	192.168.1.147	TLSv1.3	299	Application Data
6190	40.783218	192.168.1.147	192.168.1.147	TCP	44	62188 → 3030 [ACK] Seq=390 Ack=2063 Win=2159104 Len=0
6191	40.783967	192.168.1.147	192.168.1.147	TLSv1.3	86	Application Data
6192	40.783999	192.168.1.147	192.168.1.147	TCP	44	62188 → 3030 [ACK] Seq=390 Ack=2105 Win=2159104 Len=0
6193	40.784209	192.168.1.147	192.168.1.147	TCP	44	62188 → 3030 [FIN, ACK] Seq=390 Ack=2105 Win=2159104 Len=0
6194	40.784247	192.168.1.147	192.168.1.147	TCP	44	3030 → 62188 [ACK] Seq=2105 Ack=391 Win=2160896 Len=0
6195	40.784283	192.168.1.147	192.168.1.147	TLSv1.3	68	Application Data
6196	40.784313	192.168.1.147	192.168.1.147	TCP	44	62188 → 3030 [RST, ACK] Seq=391 Ack=2129 Win=0 Len=0

Frame 13: 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface \Device\NPF_{...}

Null/Loopback

Internet Protocol Version 4, Src: 192.168.1.147, Dst: 192.168.1.147

Transmission Control Protocol, Src Port: 61889, Dst Port: 3030, Seq: 0, Len: 0

The completeness of the conversation capture (tcp.completeness)

Packets: 6202 - Displayed: 6184 (99.7%) - Perdido: 0 (0.0%)

Perfil: Default

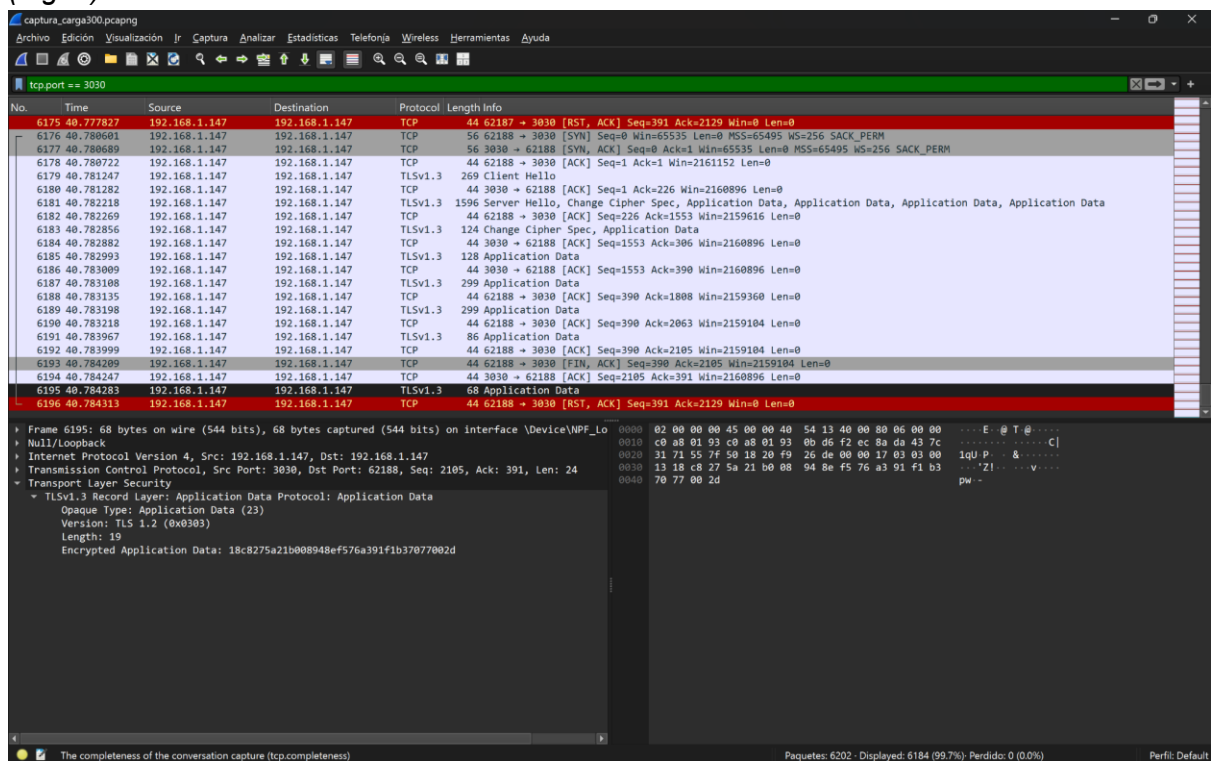
(Fig. 8)

```

Connection with ('192.168.1.147', 62184) closed
Connected by ('192.168.1.147', 62185)
Received from ('192.168.1.147', 62185): Username: test_user_297, Password: test_password_297
Connection with ('192.168.1.147', 62185) closed
Connected by ('192.168.1.147', 62186)
Received from ('192.168.1.147', 62186): Username: test_user_298, Password: test_password_298
Connection with ('192.168.1.147', 62186) closed
Connected by ('192.168.1.147', 62187)
Received from ('192.168.1.147', 62187): Username: test_user_299, Password: test_password_299
Connection with ('192.168.1.147', 62187) closed
Connected by ('192.168.1.147', 62188)
Received from ('192.168.1.147', 62188): Username: test_user_300, Password: test_password_300
Connection with ('192.168.1.147', 62188) closed

```

(Fig. 9)



(Fig. 10)